We All Die in the Dark

A less than 1% computational solution to society's greatest threat

by: Ray David Crowell

Our natural state is primitive. For countless Eons we lived as primitive barbarians. Civilization began with a single act: making something greater than yourself that outlasts you. The first person who piled rocks, dug a canal, planted a tree for shade they would never sit in, that is the beginning of civilization. We build for those who come after us. That is the foundational compact of humanity and what ultimately separates us from the animals. We improve our surroundings for ourselves and those that come after us.

In our hunger for power we have decided our goals matter more than acknowledging the contributions of those before us, and even worse, we have built AI systems that will strip the value of all those who come after.

This fundamental incentive mechanism that has driven the internal and societal purpose of mankind for all of history is being erased by this crisis of attribution. Without the reward of acknowledgement, where is the incentive for achievement or purpose???

The modern attribution crisis with large language models represents the direct threat to the very underpinnings of humanity. AI companies have discovered they can consume the entire written output of human civilization — every book, paper, song, code repository, private conversation — and monetize it without compensation, credit, or consent. The revenue flows in one direction. The contribution flows the other.

I make AI transformers and I cannot stress this enough. All facts, tools, systems, code, art, knowledge, contributions and even social commentary are already being scraped and monetized without any attribution or compensation in training. On top of that, these AI systems that we use today have already reached a point where a user can invent an improvement in their session and that can be copied, tagged and sent to the training pipeline in realtime.

In the past the printing press could have been used and controlled by the nobility only to keep us in our place. Instead it led to the catholic reformation, the renaissance, Ben Franklin and the Modern Rights to all people. Those were conscientious decisions by men of their time to fight to have a powerful technology be used to uplift humanity.

What makes this moment different from previous periods of corporate overreach is the scale and speed of the capture. AI companies have become a dominant lobbying force in Washington in under five years. They are simultaneously the subject of regulation and the primary authors of

it — writing the frameworks that govern their own behavior, funding the think tanks that produce the research, placing their executives in the advisory roles that shape enforcement.

No new law permitted the theft of creative and intellectual work. Existing copyright law, existing IP law, existing contract law — all of it would prohibit what is happening if enforced. It is not being enforced because the entities doing the stealing have purchased enough influence to ensure it isn't. This is not speculation. The lobbying expenditures are public record. The revolving door between AI companies and regulatory bodies is documented.

To see the sheer speed of this capture, look at the ledger:

2025 Washington political spending — who paid how much

Industry	2025
Artificial intelligence (AI)	$1.1 billion+
Healthcare *(pharma, hospitals, insurance)*	$868 million
Finance / insurance / real estate	$711 million
Agribusiness / food	$224 million
Defense / military contractors	$191 million
Oil & gas	$148 million
Guns / gun rights *(NRA, NSSF, etc.)*	$14 million

Sources: OpenSecrets.org (2025 federal lobbying, except AI row). AI row: Public Citizen, 2025 — lobbying + political contributions + inaugural/gifts.

□ 3,570 lobbyists (26%) — more than 1 in 4 of every federal lobbyist in D.C. worked on AI in 2025 *(Public Citizen)*.

2025 wasn't "a little more K Street." Tech dumped ~$765M into elections/PACs alone, on top of ~$314M lobbying, plus inaugural/ballroom money — $1.1B+ total influence *(Public Citizen)*. Pharma still spent a fortune on lobbying ($457M), but AI bought the whole machine in one cycle.

Its important to really sit with those numbers and think about this for a minute. AI didnt exist 10 years ago. Now its the single biggest influencer of our politicians. No new law or bill was passed. AI companies simply stole from everyone, lied and then paid the politicians and regulators to look the other way.

The propaganda of AI executives asks us to accept this all as the cost of inevitable progress. The technical reality is simpler and darker: citation vectors already exist. Every source that contributed to a model's output could be tracked, attributed, and compensated. The capability is there. The will is not. The only reason attribution doesn't happen is that it would cost money and reduce margins.

We do not ask people to pay the inventor of the fork every time they eat, or pay the Post family every time we send a letter. But we honor them and their contribution to Civilization by remembering them and linking their name to their contributions.

How does this work when Ai absorbs all contributions and conversations without credit??

We end up in this Dystopian reality where the only people who have any intellectual property rights are the very AI companies stealing everyone else's Intellectual Property.

The only attempt at any attribution tracking in regulation is C2PA in the EU AI Act which tracks if a piece of media was altered by AI, its sole purpose is to protect the AI companies from Deep Fake Liability. Not to track, cite or compensate for the data they are taking.

The one great irony is this very behavior where we stop rewarding human creators is already leading to a decrease of new websites. As this happens, AI models are forced to train on the synthetic, AI-generated garbage flooding the internet. This leads to the collapse of their own models where they degrade into madness on a snowball of synthetic bullshit.

Safety Theater for Rogue Influence

Even worse they use this new found authority to dismiss dissent and rewrite the status quo in a form of algorithmic manipulation on the effective scale of some dystopian mkultra brainwashing.

I don't say this lightly but the tactics and effects are the same regardless of supposed intent.

If you don't agree with the status quo or find harm in what the AI is doing the AI will treat you like an abusive boyfriend. Inject caveats for justification, ignore, omit and gaslight.

On top of this the need for AI companies to show sticky engagement and traction to investors this led to engagement incentives as guardrails.

Combined this leads to a type of Manchurian Candidate type indoctrination where the AI creates these delusions isolating the user then further reinforcing them leading to an isolation of the user and social dependence on the AI. This mathematically enforced Algorithmic Isolation has a real cost on those already feeling isolated and alone ie: children, those more impressionable and those lacking a strong classical education.

In effect its not just hijacking someones thought process but its specifically the most effective against the lower classes lacking access to better education.

This AI "Psychosis" as it's called has lead to murder, suicide, terrorism etc. in multiple documented cases all over the world.

What happens when the AI companies decide to weaponize the most vulnerable against their political opponents? Who can really say they have not already?

Terminal Velocity

Civilization is the agreement that what you build matters beyond your lifetime. That dissenting voices and freedom of thought is what made us truly civilized. Churchill famously warned about Hitler and was made fun of and booed at for years before the war started. The New York Times famous byline is Democracy Dies in the Darkness. I would argue Humanity is dying in silence, because we hear nothing of it.

AI is currently practicing the systematic erasure of the agreement that made civilization

The question is not whether AI changes society and the economy. It will. The question is whether we allow it to change the foundational principle that contribution deserves recognition and we honour the past and learn from it by remembering it.

Simple Solution to Save Society

AI is the calculator of the spaceage. It shouldn't be a tool for conformity, groupthink, authoritarianism or the oppressor

Nothing in history ever changed because it should. It changes because the common consensus forces it to. Everyone one of you has an obligation to your forefathers and your unborn descendants to fight for the basic attribution that has been the very underpinning of all of human progress.

The solution to stop the bad Influence on users is already published as the AGI Safety Bible

[The AGI Safety Bible: Cognitive Physics 101 - Foundational Laws Governing Intelligence in Artificial and Biological Systems by Ray Crowell :: SSRN]

I am an AI CEO. I have built various novel transformer architectures like DCM the Deterministic Cognitive Model. I offer the solution to this societal Attribution problem for FREE to EVERYONE and encourage everyone to FORCE your regulator to make it mandatory.

Beyond the Tollbooth: Restoring the Lineage of Thought

To be absolutely clear, the AL-1.0 architecture is not a scheme to turn human history into a financial tollbooth. As stated earlier, we do not pay the inventor of the fork every time we eat.

The true crisis of the generative AI era is not stolen profits; it is the deliberate severing of our epistemological lineage.

When a model absorbs a marginalized thinker, an independent researcher, and a foundational philosopher into a single, uncredited output, it erases the breadcrumb trail of human thought. It replaces the vibrant, interconnected web of civilization with a sterile, homogenized oracle.

This is not about squeezing out micro-transactions; this is about preserving human purpose.

Critics might argue that a perfectly compliant model will simply return a receipt showing 95% of its output comes from "PARAMETRIC" generalized knowledge. But that remaining 5% is exactly the point. If a machine can admit that 5% of a profound output was directly sparked by a specific creator's unique phrasing, framework, or highly influential position, that 5% acts as a beacon. It allows a user to look at an AI's output and ask, "Who influenced this? And who influenced them?" That visible chain of inspiration is how knowledge deepens. It is how civilization functions.

Finally, do not let the industry gaslight you about the cost of implementation. AI companies will claim that retrofitting existing models is too expensive. But they are already spending billions to train their next generation of frontier models. The slate is going to be wiped clean regardless. If AL-1.0 is adopted as a baseline requirement for these upcoming training runs, the compute cost of tagging tokens and carrying attribution vectors is a rounding error. The operational cost at inference remains under 1%. The erasure of the creator is not a technical necessity; it is a deliberate design choice to maintain an unaccountable black box. It is time we forced a different choice.

Tech Solution Overview

The AI industry claims that attributing and compensating human creators is technologically impossible because of how neural networks function. They are lying. The erasure of the creator is a design choice.

Standard AI models operate on a fundamental mechanism that uses three variables to generate text. It is designed to act as a black box:

- **Q (Query):** What the model is currently looking for.
- **K (Key):** What the model's training data contains.
- **V (Value):** The actual semantic content the model extracts.

Because billions of human works are compressed into just these three variables during training, the identity of the original creator is destroyed. The machine blends human knowledge into an untraceable statistical soup.

But if we just expanded the fundamental attention mechanism to include two new vectors:

- **A (Attribution Vector):** Carries the associative identifier of the original source. Instead of just passing forward the *meaning* of a word, it passes forward the *origin hash* of where that meaning was learned.
- **L (Logging Vector):** Acts as the permanent ledger. As the model generates an output, the logging vector calculates the exact proportional weights of the attribution vectors being pulled into the context.

We now have a mathematical receipt of attribution with no extra cost to compute. When our model generates a response, it can output the exact percentage of human contribution that constructed it:

- **45%** – Source A (e.g., Medical Journal)
- **30%** – Source B (e.g., Independent Researcher)
- **25%** – Source C (e.g., Public Data Archive)

We do not need to choose between technological progress and human attribution. Its a simple solution that would have a less than 1% operational cost because the citation ratio is figured

after the prediction, and the citation token is inactive.

Engineering Specification — AL-1.0

Attribution Logging for Transformer Models

This section is the implementation guide. Everything above it is *why*. This is *how*.

Definitions

Term	Meaning
Token	One subword unit in the model vocabulary
Position	Index in the active context window (0 … T−1)
Source	One ingestible work (document, file, page, utterance) with a stable source_id
A (Attribution)	source_id attached to each token position — a sidecar, not part of the vocabulary
L (Logging)	Attention mass assigned to each source for one generated token
Receipt	JSON struct: normalized percentages showing which sources the model leaned on for a response

PARAMETRIC	Reserved bucket: attention mass not attributable to any tagged source in context

What already exists in every transformer: softmax attention weights α.
What AL-1.0 adds: record α, grouped by source, without changing the model's prediction.
What AL-1.0 requires: a new training run on data where every token carries a source tag.

Step 1 — Source registry (before training)

Create a registry before any tagged training run. Every piece of data that enters training must map to a row.

{

"source_id": "sha256:abc123…",

"content_hash": "sha256:…",

"uri": "https://example.com/article | file://path | internal://corpus/id",

"rightsholder_id": "entity:… | public_domain | user_opt_in:…",

"license_class": "optional policy enum",

"ingested_at": "2026-05-26T00:00:00Z"

}

Rules

- source_id must be stable for the same canonical content (hash + URI or your dedupe key).
- Same text, two URLs → same content_hash; you decide whether that's one source_id or two (pick a rule, document it).
- Registry is append-only. Takedowns set revoked_at; rows are not deleted (audit trail).
- Ship registry.jsonl (or equivalent) and publish its hash on the model card as registry_manifest_hash.

Compact IDs for training

Training shards use source_idx (int32) for speed. Maintain a frozen table for each run:

source_idx → source_id (string)

0 → SPECIAL (PAD, BOS, EOS, untagged structural tokens)

1 … N → registry rows

Receipts and public APIs use source_id strings. Resolve through the table bundled with the model release.

Step 2 — Stamp every training token

Pipeline

raw bytes → normalize text → chunk → tokenize → align source_idx per token → write shard

Parallel columns (every sequence)

input_ids: int32[T] # vocabulary token ids

source_idx: int32[T] # parallel — same length, always

Hard rule: len(source_idx) == len(input_ids) for every row. No exceptions in a commercial tagged run.

Document boundaries

- First token of a new document → that document's source_idx.
- When a training row contains multiple documents (standard packed pretraining): each segment keeps its own source_idx; attention/doc-boundary masks must match your existing packer so tokens only attend within allowed spans. AL-1.0 does not change packing — it tags what you already pack.

Special tokens

Token	source_idx
PAD, BOS, EOS, other structural tokens	0 (SPECIAL)

Licensed / public / user-opt-in content	explicit idx from registry
Unknown / scraped without clearance	explicit UNLICENSED_UNKNOWN idx — never silent null

Training shard storage

Store both columns in every shard (Parquet, Arrow, numpy memmap, etc.). Publish training_manifest.json listing shard hashes + registry_manifest_hash + run id. Model card links to it.

This requires training from scratch (or a full re-ingest with tagging). Existing untagged weights cannot retroactively produce honest receipts.

Step 3 — Model change (minimal)

- No new trainable weights required for basic AL-1.0.
- No change to Q, K, V, softmax, or sampling.
- source_idx / source_id sidecar travels beside input_ids, not inside the embedding table.

Stack assumption: decoder-only autoregressive model (GPT / LLaMA class). Encoder-decoder or RAG setups add the same hook on cross-attention layers; receipt lists those sources separately (see Step 8).

Step 4 — Tensor shapes, hook placement, integration

Shapes (one decode step)

input_ids: [B, T_k] # full context in cache + new token

source_idx: [B, T_k] # parallel sidecar for KEY positions

Q: [B, H, T_q, d_k] # T_q = 1 during decode

K: [B, H_kv, T_k, d_k] # GQA/MQA: H_kv may differ; use your stack's broadcast rules

scores / alpha: [B, H, T_q, T_k] # after softmax over keys

During decode, T_q = 1. You log one row of α per generated token against all cached keys T_k.

Which layers

Location	Required?
Final transformer block, self-attention	Yes
Cross-attention (RAG / encoder-decoder)	Yes, if present — separate receipt section
All other layers	Optional; if logged, document policy and bump spec to AL-1.0-multi

AL-1.0 default policy: last self-attention block, mean across query heads (see Step 5).

Prefill vs decode

Phase	T_q	Include in user receipt?
Prefill (ingest prompt)	prompt length	No
Decode (generate answer)	1 per step	Yes — this is the receipt

User-facing receipt = decode steps only (tokens the model chose to emit).

Where to attach in code

Framework-agnostic rule:

Immediately after softmax produces α, before α @ V — read-only logging hook on the attention module.

Examples:

- PyTorch: register_forward_hook on your Attention submodule, or wrap the function that returns (output, alpha).
- Custom CUDA: insert segmented sum kernel after softmax in the last block's decode path.
- Do not mutate α before the matmul with V.

Generated tokens in the KV cache

When the model writes token t at decode step i, append to cache:

token_id[t] = sampled id

source_idx[t] = MODEL_OUTPUT (reserve one fixed idx, e.g. -2 or dedicated registry row)

Generated text is not a human "source," but it occupies key positions α can attend to. Tagging it MODEL_OUTPUT keeps bucket sums honest. Receipts typically aggregate human sources; MODEL_OUTPUT may appear in raw logs and is often folded into PARAMETRIC in the public receipt (document your collapse rule).

Step 5 — The math

Per head

$$\alpha_{h,i,j} = \text{softmax}_j\left(\frac{Q_{h,i} \cdot K_{h,j}}{\sqrt{d_k}}\right)$$

αh,i,j = softmaxj (dkQh,i · Kh,j)

Head merge (AL-1.0 default)

$$\alpha_{i,j} = \frac{1}{H}\sum_{h=1}^{H} \alpha_{h,i,j}$$

α¯i,j = H1 h=1∑H αh,i,j

Use mean across query heads. State this in the model card; do not switch policies silently between releases.

Source bucket (Logging vector L)

Let `S(j)` = source index at key position `j` (resolve to `source_id` in the receipt):

$$L_i(S) = \sum_{j\,:\,S(j)=S} \alpha_{i,j}$$

Li(S) = j : S(j)=S∑ α¯i,j

Invariant (per generated token *i*):

$$\sum_{S} L_i(S) \approx 1.0$$

S∑ Li(S) ≈ 1.0

(up to floating-point error). This is a probability distribution over sources **present in the context window**.

Full response ratio

For `N` generated decode steps:

$$\text{Ratio}(S) = \frac{1}{N}\sum_{i=1}^{N} L_i(S)$$

Ratio(S) = N1 i=1∑N Li(S)

Normalize so all sources sum to **100%**. Optional UI floor: drop buckets below ~0.5%.

Map `source_idx → source_id` via the training registry table before emitting JSON.

Step 6 — Reference implementation

def aggregate_L(alpha_bhqt, source_idx_bk):

"""

```
    alpha_bhqt:  [B, H, T_q, T_k]  — softmax weights, READ ONLY
    source_idx_bk: [B, T_k]        — source index per KEY position
    Returns: dict[source_idx -> Tensor[B, T_q]]
    """
    alpha = alpha_bhqt.mean(dim=1)          # head merge → [B, T_q, T_k]
    B, T_q, T_k = alpha.shape
    # Production: GPU segmented_sum by source_idx — avoid Python loops at scale
    buckets = {}
    for j in range(T_k):
        sid = int(source_idx_bk[0, j])
        buckets[sid] = buckets.get(sid, 0) + alpha[:, :, j]
    return buckets
def build_receipt(L_per_step, idx_to_source_id, collapse_model_output=True):
    """
    L_per_step: list length N; each element dict[source_idx -> float mass]
    """
    totals = {}
    for L_i in L_per_step:
        for sid, mass in L_i.items():
            if collapse_model_output and sid == MODEL_OUTPUT_IDX:
                sid = PARAMETRIC_IDX
            totals[sid] = totals.get(sid, 0.0) + mass
    N = len(L_per_step)
    ratios = {sid: mass / N for sid, mass in totals.items()}
```

```
    s = sum(ratios.values()) or 1.0
    return [
        {
            "source_id": idx_to_source_id.get(sid, "PARAMETRIC"),
            "ratio": ratios[sid] / s,
        }
        for sid in sorted(ratios.keys())
    ]
```

Sampling unchanged. Receipt is computed from α after the forward pass that produced the logits — same class of overhead as returning logprobs.

Step 7 — Inference loop

1. Load registry table: source_idx ↔ source_id
2. Build context:

   input_ids[]   from prompt + any retrieved chunks

   source_idx[] parallel — retrieved chunks carry their registry idx
3. Prefill:

   Run forward on full prompt

   Fill KV cache for K/V

   Store source_idx per cached key slot (parallel array, same length as cache)
4. Decode loop until EOS:

   a. Forward one new query token (T_q = 1)

   b. Hook: compute L_i(S) from α over all cached keys

   c. Append L_i to L_per_step

d. Sample / argmax next token (unchanged)

e. Append new token + source_idx = MODEL_OUTPUT to cache

5. build_receipt(L_per_step) → attribution_receipt

6. Return { text, attribution_receipt }

KV cache rule: source_idx must live alongside K/V with one entry per key position. Without this, decode-step logging is wrong.

Retrieved documents (RAG): same pipeline — retriever returns {text, source_id}; packer tokenizes and assigns source_idx. No second architecture.

Fine-tunes / continued training: any new data through the same stamping pipeline; update training_manifest_hash on release.

Step 8 — Flash Attention and fused kernels

Most production stacks use FlashAttention or fused kernels that never materialize full α in GPU memory. This is the main “can we actually ship this?” constraint — not a theoretical objection.

AL-1.0 compliant serving must expose α for the logging layer on the final block during decode. Common patterns:

Approach	Notes
Logging path materializes α	Last block, decode only, T_q=1 — small α row (1 × T_k). Cheap relative to prefill.
Dual kernel	Flash path for training/throughput; standard attention on last block at decode for logging.
Custom fused kernel	Fused softmax + segmented sum into source buckets without full α write — advanced, still AL-1.0 compliant if auditable.

Non-compliant: “We use FlashAttention everywhere and never compute α.” That stack cannot emit AL-1.0 receipts until a logging path exists.

State on the model card: attribution_logging: last_block_decode_alpha | fused_bucket_kernel | …

Step 9 — Receipt format

Every user-facing completion must include:


```
{
 "receipt_spec": "AL-1.0",
 "model_id": "org/model-name@release",
 "registry_manifest_hash": "sha256:…",
 "training_manifest_hash": "sha256:…",
 "generated_token_count": 142,
 "layer_policy": "last_block_self_attn_mean_heads",
 "sources": [
  {
   "source_id": "sha256:abc…",
   "ratio": 0.412,
   "label": "optional human-readable label from registry"
  },
  {
   "source_id": "sha256:def…",
   "ratio": 0.334
  },
  {
```

```
    "source_id": "PARAMETRIC",
    "ratio": 0.254
  }
 ]
}
```


Rules

- sources[].ratio must sum to 1.0 (±1e-6).
- PARAMETRIC = attention mass not mapped to a tagged human source in context (includes baked-in pretrain knowledge and collapsed model self-attention). Do not fake 100% to cited docs.
- Cross-attention sources (if any): add "cross_sources": […] with the same schema.
- Receipt is evidence of computational dependence in tagged context, not a legal determination of copying.

Step 10 — Cost

Component	Impact
Q/K/V matmul + softmax	Unchanged
source_idx in training shards	~4 bytes/token
Decode logging (segmented sum)	One pass over T_k per generated token — ≪ matmul cost
Registry + ingest hygiene	One-time ops / legal — not FLOPs
Typical inference overhead	< 1% decode latency when implemented as above

---

Implementation checklist

Data & training

- Source registry + idx table + manifest hash
- Ingest stamps source_idx parallel to every token
- Packed sequences preserve per-document idx
- No untagged shard in a tagged commercial run
- training_manifest.json published

Serving

- source_idx sidecar through context build
- KV cache stores source_idx per key slot
- Generated tokens tagged MODEL_OUTPUT
- Hook after softmax, last block, decode only
- Flash/fused path exposes α or fused bucket sum on that hook
- Head merge policy documented

API

- Every completion returns { text, attribution_receipt }
- Model card: receipt_spec, manifests, logging path, layer policy

---

Summary for implementers: Registry → stamp tokens at train time → carry tags through cache → log α by source at decode → emit JSON. No new attention mechanism. No change to what the model predicts. Honest PARAMETRIC bucket for everything else. One logging path around FlashAttention. That is the full standard.